

Análisis de parámetros de un conversor ADC

Universidad Militar Nueva Granada – Ingeniería en Telecomunicaciones
Miguel Ángel Plazas – Daniel Mateo Alegría Bernate
Docente: José de Jesús Rúgeles Uribe

Resumen -En esta práctica se estudió experimentalmente el proceso de conversión analógico-digital utilizando una Raspberry Pi Pico 2W basada en el microcontrolador RP2350. En la primera parte se analizó el proceso de muestreo de una señal senoidal de 250 Hz, determinando la frecuencia y el período de muestreo, así como el número de muestras obtenidas por período. En la segunda parte se evaluó el comportamiento dinámico del ADC mediante el procesamiento espectral de las muestras utilizando la Transformada Rápida de Fourier (FFT), variando el número de puntos de 128 a 4096. Se analizaron parámetros como resolución espectral, piso de ruido, SNR, THD, SINAD, ENOB y SFDR, comparando los resultados experimentales con los valores teóricos correspondientes a un ADC ideal de 12 bits. Finalmente, se identificaron las limitaciones prácticas de la plataforma al aumentar el tamaño de la FFT debido al consumo de memoria durante el procesamiento.

Palabras clave - ADC, conversión analógico-digital, muestreo, Raspberry Pi Pico 2W, RP2350, resolución, exactitud, repetibilidad.

Abstract — In this experiment, the analog-to-digital conversion process was studied using a Raspberry Pi Pico 2W based on the RP2350 microcontroller. The first part analyzed the sampling process of a sinusoidal signal, determining the sampling frequency and period, as well as the number of samples obtained per period for various input frequencies. The second part involved a statistical evaluation of the ADC's behavior using five DC voltage levels and 10,000 measurements per trial.

Keywords— ADC, analog-to-digital conversion, sampling, Raspberry Pi Pico 2W, RP2350, resolution, accuracy, repeatability

I. INTRODUCCIÓN

La conversión analógico-digital (A/D) es un proceso fundamental en los sistemas de comunicación digital, ya que permite transformar señales analógicas en valores digitales que pueden ser procesados por sistemas electrónicos. Este proceso se basa en la toma de muestras de una señal continua a determinados intervalos de tiempo, por lo que la frecuencia de muestreo y el número de muestras obtenidas por período son parámetros importantes para representar adecuadamente la señal. En esta práctica de laboratorio se exploró

experimentalmente el proceso de conversión A/D utilizando una Raspberry Pi Pico 2 W, basada en el microcontrolador RP2350. Inicialmente, se verificó la frecuencia de muestreo del sistema mediante el análisis temporal de la señal, comparando el valor establecido en el programa con la frecuencia observada mediante el osciloscopio. La guía establece como referencia una señal de entrada de 250 Hz y una frecuencia de muestreo objetivo de 2000 Hz, con aproximadamente ocho muestras por ciclo. Posteriormente, se analizó el comportamiento espectral de las muestras obtenidas mediante la Transformada Rápida de Fourier (FFT), modificando el número de muestras utilizadas para observar su efecto sobre la resolución en frecuencia y el piso de ruido. Para ello, se consideraron diferentes tamaños de FFT, desde 128 hasta 4096 puntos, permitiendo estudiar experimentalmente la ganancia de procesamiento asociada al aumento del número de puntos.

II. OBJETIVOS

A. Objetivo General

Evaluar experimentalmente el desempeño de un convertidor A/D de 12 bits mediante la caracterización de la frecuencia y estabilidad de muestreo, el análisis de parámetros dinámicos y el efecto de la FFT sobre el piso de ruido, contrastando los resultados obtenidos con los límites teóricos establecidos para un ADC ideal según la nota de aplicación AN693 de Microchip.

B. Objetivos Específicos

- Determinar experimentalmente la frecuencia y estabilidad de muestreo (F_s) en un microcontrolador usando osciloscopio e instrumentación digital.
- Evaluar el desempeño dinámico del convertidor A/D cuantificando los parámetros clave: SNR, THD, SINAD, ENOB SFDR y FFT Noise Floor.
- Demostrar cuantitativa y visualmente la ganancia de procesamiento (processing gain) de la FFT y el descenso del piso de ruido por bin al variar el número de puntos M (muestras N).
- Contrastar los resultados experimentales frente a los límites teóricos del ADC ideal de 12 bits según la nota de aplicación Microchip AN693.

III. MARCO TEÓRICO

A. Conversión analógico-digital

La conversión analógico-digital (A/D) es el proceso mediante el cual una señal analógica continua se transforma en una representación digital. Este proceso comprende las etapas de muestreo y cuantización, permitiendo representar los valores de una señal mediante códigos digitales. [1].

La resolución del convertidor determina la cantidad de niveles disponibles para representar la amplitud de la señal [1].

B. Muestreo

El muestreo consiste en obtener valores de una señal continua en intervalos de tiempo determinados. [2]. El período de muestreo T_s corresponde al intervalo entre dos muestras consecutivas y se relaciona con la frecuencia de muestreo F_s mediante:

$$T_s = \frac{1}{F_s}$$

De manera equivalente:

$$F_s = \frac{1}{T_s}$$

C. Raspberry Pi Pico 2W y ADC

La Raspberry Pi Pico 2 W es una placa de desarrollo basada en el microcontrolador RP2350. Este microcontrolador incorpora un convertidor analógico-digital (ADC) de 12 bits, utilizado para convertir señales analógicas en valores digitales. El ADC dispone de entradas analógicas asociadas a determinados pines GPIO, entre ellas GP26, correspondiente al canal ADC0 [1].

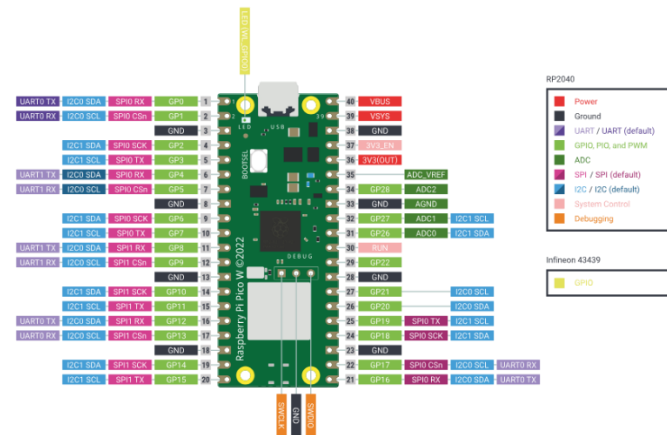


Figure 1. Raspberry Pi Pico 2W

En la Figura 1 se identifican los pines utilizados para la adquisición y visualización del proceso de muestreo. El pin **GP26/ADC0** corresponde a la entrada de la señal analógica proveniente del generador de funciones, mientras que **GND** proporciona la referencia común entre los equipos.

El pin **GP15** corresponde a la salida digital utilizada para indicar los instantes de adquisición. De esta manera, la señal analógica se encuentra asociada a la entrada del ADC y la señal digital permite identificar temporalmente las muestras realizadas.

D. Muestras por período

El número de muestras obtenidas durante un período de una señal depende de la relación entre la frecuencia de muestreo y la frecuencia de la señal:

$$N_s = \frac{F_s}{f_{in}}$$

donde N_s representa el número de muestras por período, F_s la frecuencia de muestreo y f_{in} la frecuencia de la señal.

También puede expresarse mediante los períodos:

$$N_s = \frac{T_{in}}{T_s}$$

Por consiguiente, manteniendo constante la frecuencia de muestreo, un aumento en la frecuencia de la señal produce una disminución del número de muestras disponibles para representar cada período.

E. Teorema de Nyquist

El teorema de Nyquist-Shannon establece que una señal limitada en banda puede ser reconstruida correctamente a partir de sus muestras cuando la frecuencia de muestreo es superior al doble de la máxima frecuencia presente en la señal [2]:

$$F_s = 2 f_{max}$$

A partir de esta condición se define la frecuencia de Nyquist como:

$$f_N = \frac{F_s}{2}$$

F. Resolución del ADC

La resolución de un convertidor analógico-digital determina el número de niveles de cuantización que puede utilizar para representar una señal. Para un ADC de N bits, el número de niveles está dado por:

$$L = 2^N$$



El RP2350 dispone de un ADC con resolución nominal de 12 bits, por lo que:

$$L = 2^{12} = 4096$$

Los códigos digitales se encuentran entre 0 y:

$$2^{12} - 1 = 4095$$

La resolución ideal en tensión, o tamaño de un LSB, puede calcularse mediante:

$$LSB = \frac{V_{REF}}{2^N - 1}$$

Para $V_{REF}=3,3\text{ V}$

$$LSB = \frac{3.3\text{ V}}{4095} = 0.806\text{ mV}$$

Esto significa que, idealmente, cada incremento del código digital representa aproximadamente 0,806 mV dentro del rango de referencia establecido

G. Escala completa y dBFS

La función “read_u16()” de MicroPython entrega valores entre 0 y 65535. Sin embargo, esto no significa que el ADC posea una resolución física de 16 bits. La conversión realizada por el ADC del RP2350 continúa siendo de 12 bits y posteriormente la lectura es representada mediante una escala de 16 bits [1][3].

La relación indicada para recuperar el código original de 12 bits es:

$$Code_{12} = raw16 \gg 4$$

Donde raw16 corresponde al valor obtenido mediante read_u16().

Finalmente, para convertir una lectura de read_u16() en una tensión aproximada puede utilizarse: [3].

$$V_{in} = \frac{raw16}{65535} \times V_{REF}$$

Cualquier nivel RMS medido se expresa en dBFS mediante:

$$L_{dBFS} = 20 \log_{10} \left(\frac{V_{RMS}}{V_{FS,RMS}} \right)$$

H. SNR ideal de cuantización

Relacionando la potencia RMS de la señal a escala completa con la potencia del ruido de cuantización puro definido en la Sección F, se obtiene la ecuación del SNR ideal de un

convertidor de Nb bits, presentada en AN693 (Ecuación 3) [6], [7]:

$$SNR_{ideal} = 10 \log_{10} \left(V_{FS}, \frac{RMS^2}{V_q, RMS^2} \right) = 6,02nB + 1,76$$

Para Nb = 12 bits nominales, $SNR_{ideal} \approx 74,00\text{ dB}$, valor tomado como referencia teórica para contrastar el desempeño dinámico medido experimentalmente.

I. Resolución espectral, longitud de la FFT y zero padding

Al adquirir N muestras a una tasa Fs, la resolución física irreducible del registro temporal es $\Delta f_{FFT} = \frac{F_s}{M}$ [7]. Si se calcula una FFT de longitud M = NFFT mayor que N rellenando con ceros (zero padding), el espaciamiento gráfico entre bins pasa a ser $\Delta f_{FFT} = \frac{F_s}{M}$; no obstante, el zero padding actúa únicamente como una interpolación visual de la Transformada de Fourier en Tiempo Discreto (DTFT) y no incrementa la resolución física del análisis ni aporta nuevas muestras estadísticamente independientes [7]. En las pruebas realizadas no se empleó zero padding, de modo que M = Ns para todos los valores evaluados (128, 256, 512, 1024, 2048 y 4096).

J. Definición de Métricas Dinámicas

Separando la potencia de la componente fundamental (P_{signal}), de los armónicos (P_h) y del ruido (P_n), AN693 define las métricas dinámicas empleadas para caracterizar el desempeño real del convertidor [6], [7]:

$$SNR = 10 \log_{10} \left(\frac{P_{signal}}{P_n} \right)$$

$$THD = 10 \log_{10} \left(\frac{P_h}{P_{signal}} \right)$$

$$THD + N = 10 \log_{10} \left(\frac{P_h + P_n}{P_{signal}} \right)$$

$$SINAD = 10 \log_{10} \left(\frac{P_{signal}}{P_n + P_h} \right) \approx -(THD + N_{dB})$$

$$ENOB = \frac{SINAD - 1,76}{6,02}$$

$$SFDR = 20 \log_{10} \left(\frac{A_{fund}}{A_{spur,max}} \right)$$

K. Jitter Temporal y Tensión de Referencia

Si la base de tiempos del sistema de adquisición presenta una desviación estándar de jitter σ_t , el límite superior de la relación señal a ruido alcanzable para una entrada senoidal de frecuencia fin está dado por [6], [7]:

$$SNR_{jitter} \approx -20 \log_{10}(2\pi f_{in} \sigma_t)$$

Por otro lado, cualquier desviación en el valor real de VREF introduce un error de escala lineal sobre las magnitudes absolutas de tensión; sin embargo, las métricas espectrales relativas (SNR, THD, SINAD, SFDR) son cocientes normalizados de potencia y resultan, por tanto, inmunes a variaciones leves en la referencia de tensión continua [6], [7]. Esta propiedad justifica que las comparaciones se centren en dichas métricas relativas y no en los niveles absolutos de tensión.

IV. RESULTADOS EXPERIMENTAL

I. Dominio del Tiempo y Verificación del Muestreo (Fs)

En esta fase se implementó el montaje de adquisición y visualización con la Raspberry Pi Pico 2W conectada a un generador de señales y un osciloscopio digital ejecutando el programa ADC_testing.py. ver ANEXO A.

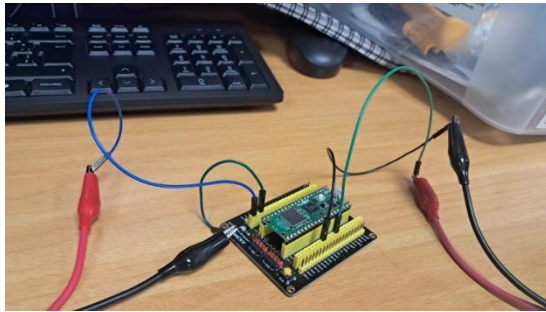


Figure 2. Montaje

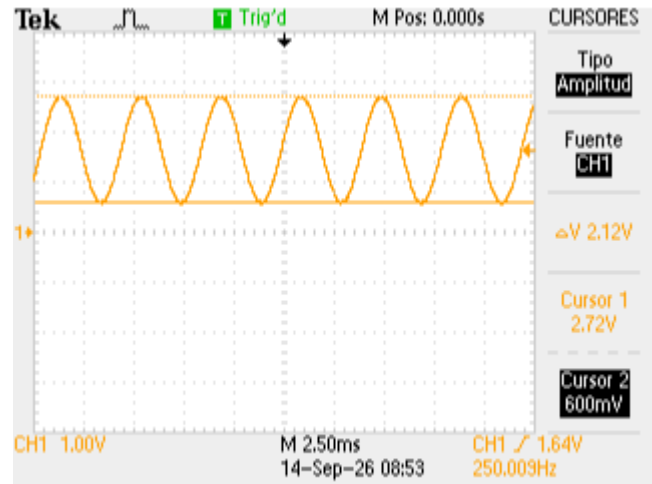
Podemos observar como la señal del generador se conecta a la entrada GP27/ADC0 de la Raspberry Pi Pico 2 W y al canal 1 del osciloscopio, mientras que GP15 se conecta al canal 2 para visualizar los instantes de muestreo.

A. Preparación de la Señal de Entrada

Inicialmente, se verificó que la señal de entrada se encontrara dentro del rango de tensión permitido por la entrada ADC. Para ello, el generador de funciones se configuró con los siguientes parámetros:

- Señal senoidal
- $f = 250 \text{ Hz}$
- $V_{pp} = 2.0 \text{ V}$
- Offset = 1.65 V.

Antes de conectar el generador a la Raspberry Pi Pico 2W, la señal se observó en el canal 1 (CH1) del osciloscopio para comprobar los valores máximos y mínimos de la señal.



TDS 2012C - 9:03:57 a. m. 14/09/2026

Figure 3 .Verificación inicial de la señal senoidal

A partir de esta medición se registraron los valores máximo y mínimo obtenidos permiten verificar que la señal permanece dentro del rango de 0 a 3,3 V requerido por la entrada ADC.

Una vez identificado los datos se procedió a comparar los resultados se presentan en la Tabla 1.

Vmax

$$\%E = \frac{|2,72 \text{ V} - 2,65 \text{ V}|}{2,65 \text{ V}} \times 100 = 2,64\%$$

Vmin

$$\%E = \frac{|600 \text{ mV} - 650 \text{ mV}|}{650 \text{ mV}} \times 100 = 7,69\%$$

Magnitud	Teórico	Exp	% Error
Vmax	2,65 V	2,72 V	2,64%
Vmin	650 mV	600 mV	7,69%

Tabla 1.Comparación teórico/experimental de Vmax y Vmin

La comparación muestra que el valor máximo experimental fue de 2,72 V, frente a los 2,65 V esperados, lo que corresponde a un error de 2,64 %. Para el valor mínimo se obtuvo 0,60 V, frente a los 0,65 V teóricos, con un error de 7,69 %.

Aunque existen diferencias entre los valores configurados y los medidos, la señal permanece completamente dentro del intervalo de 0 a 3,3 V. Por tanto, las condiciones de amplitud y offset fueron adecuadas para continuar con la conexión al ADC

Las diferencias observadas pueden atribuirse a la tolerancia y precisión de los instrumentos y del generador de funciones, además de las condiciones de medición

B. Adquisición y verificación de la frecuencia de muestreo

Una vez verificada la señal de entrada, se ejecutó el programa **ADC_testing.py**, utilizando los parámetros establecidos por defecto para la prueba: una frecuencia de entrada de **250 Hz** y la ventana **Hann**.

Se realizaron cinco mediciones para obtener un promedio de datos para posteriormente compararlos, a continuación, se muestra la primera captura, Para ver todas las imágenes diríjase al **ANEXO B**

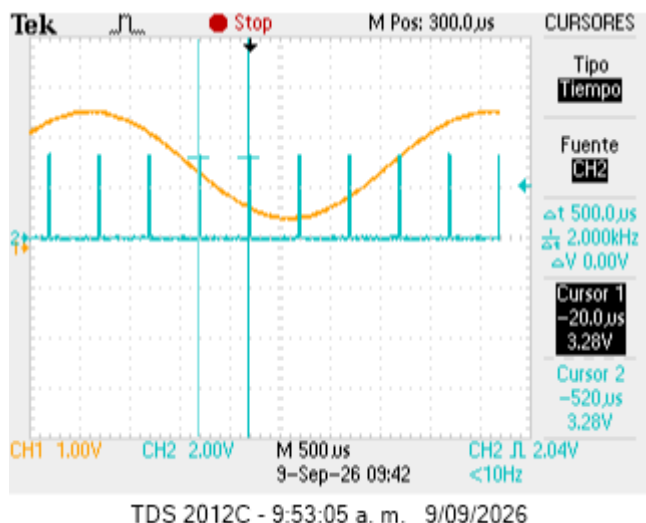


Figure 4. Medición de los instantes de muestreo

A través de la medición con cursores en el Canal 2 (CH2) del osciloscopio, pudimos determinar el intervalo entre muestras sucesivas (Δt) = 500 μ s y la frecuencia de muestreo real en hardware $F_s^{(osc)} = 2000 \text{ Hz}$ que nos proporcionaba el GP15

C. Comparación con la frecuencia calculada por software

Al finalizar la adquisición, el programa presentó en la consola una estimación de la frecuencia de muestreo y otros parámetros asociados a los intervalos entre muestras. La información completa se encuentra en el **ANEXO C**

```
--- Sampling ---
Target Fs      : 2000.000 Hz
Measured Fs    : 2000.352 Hz
Fs error       : +0.0176 %
Mean Ts        : 499.912 us
Timing jitter (std) : 2.509 us
DC offset      : 1.647358 V
```

Figure 5. Valores consola

Podemos observar que la frecuencia estimada por el software es de 2000,352 Hz, valor que presenta una diferencia muy pequeña con respecto a la frecuencia nominal de 2000 Hz y a la medición realizada mediante los cursores del osciloscopio.

Una vez identificados los datos, se procedió a comparar los resultados, los cuales se presentan en la Tabla 2.

$$\%E = \frac{|2000,352 \text{ Hz} - 2000 \text{ Hz}|}{2000 \text{ Hz}} \times 100 = 0.0176\%$$

Fs Teo	Fs Software	Fs Osc	Timing jitter
2000Hz	2.000,352 Hz	2000 Hz	2,509us

Tabla 2. Comparación (Fs) teórica, software, osciloscopio y timing jitter

La diferencia obtenida entre el valor nominal y la frecuencia estimada por software es de aproximadamente 0,0176 %, lo que representa una variación muy pequeña y muestra una alta correspondencia entre la frecuencia de muestreo programada y la obtenida durante la adquisición.

D. Medición de la frecuencia de la señal analógica

A continuación, mediante los cursores del canal 1 (CH1), se midió el período de la señal senoidal de entrada.

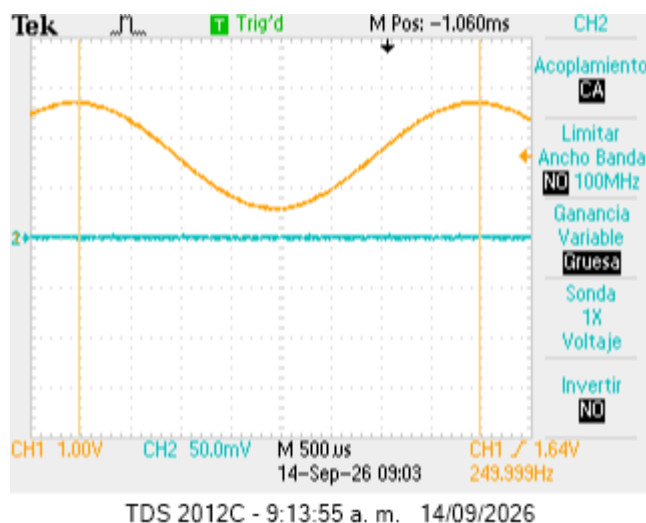


Figure 6. Periodo CH1

De acuerdo con la medición realizada en el osciloscopio, se obtuvo un período de entrada de $T_{in} = 4 \text{ ms}$. A partir del período medido, la frecuencia de la señal de entrada se calculó como:

$$f_{in} = \frac{1}{4 \text{ ms}} = 250 \text{ Hz}$$

Con la frecuencia de muestreo y la frecuencia de entrada determinadas experimentalmente, se calculó el número de muestras que toma por período de la señal.

$$N_s = \frac{2000 \text{ Hz}}{250 \text{ Hz}} = 8 \text{ muestras / periodo}$$

Al ejecutar la señal en el osciloscopio pudimos obtener una confirmación con respecto a lo calculado

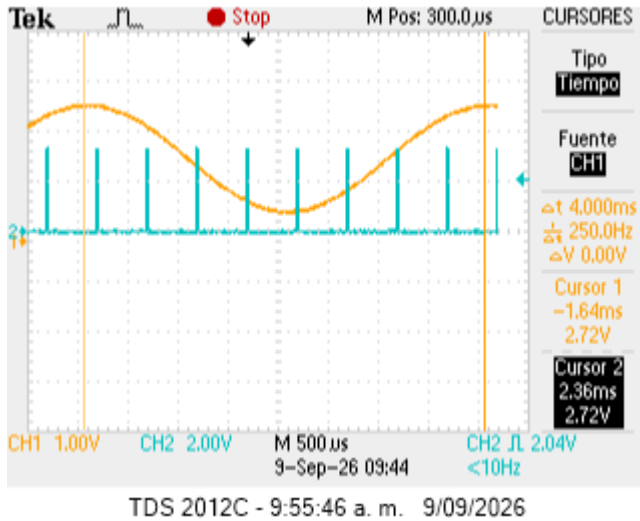


Figure 7.Verificación de las muestras por ciclo

Podemos observar como el número de flancos ascendentes de CH2 que caen exactamente dentro de un ciclo completo de la onda senoidal en CH1, fue exactamente 8 pulsos, confirmando de la teoría del muestreo.

II. Procesamiento Espectral y Piso de Ruido

En esta segunda parte de la práctica se analizó experimentalmente el desempeño dinámico del convertor A/D de la Raspberry Pi Pico 2W mediante análisis espectral (FFT), ejecutando el programa ADC_testing.py con distintos valores del número de muestras capturadas (N_s) y de la longitud de la FFT ($M = NFFT$).

A. Montaje experimental y adquisición de las mediciones

Se mantuvo el mismo montaje físico utilizado en la Fase 1. La salida del generador de funciones se conectó a la entrada **GP27/ADC0** de la Raspberry Pi Pico 2W y al canal 1 (CH1) del osciloscopio. El pin GP15 se conectó al canal 2 (CH2) del osciloscopio para visualizar los instantes de muestreo.

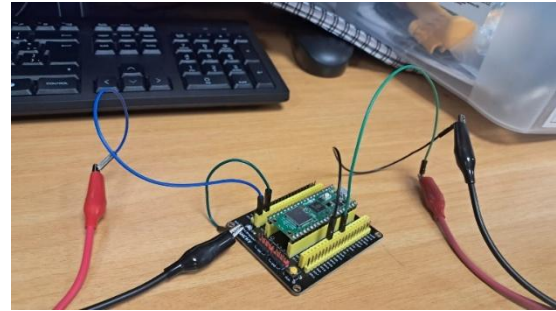


Figure 8. Montaje experimental

Para ello, el generador de funciones se configuró con los siguientes parámetros:

- Señal senoidal
- $f = 250 \text{ Hz}$
- $V_{pp} = 2.6 \text{ V}$
- Offset = 1.65 V.

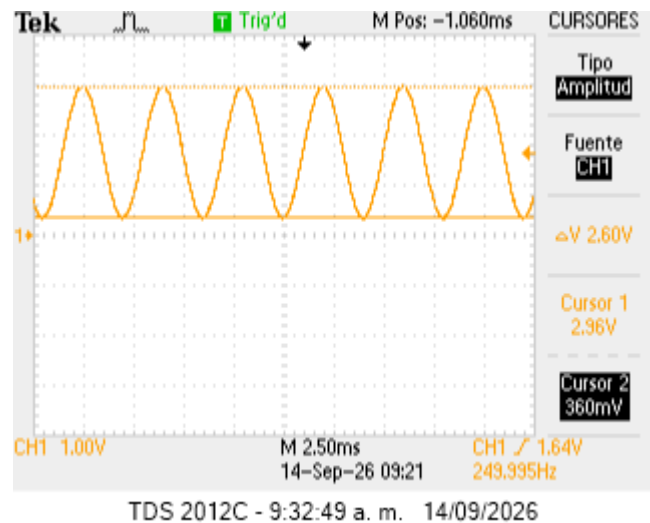


Figure 9 Verificación Voltaje

Para cada prueba se ejecutó ADC_testing.py seleccionando siempre la ventana Hann por defecto y modificando en el script las variables N_s y $NFFT$ antes de cada corrida.

B. Ejecución y limitaciones de las pruebas

Se realizaron las pruebas correspondientes a diferentes cantidades de muestras, aumentando progresivamente el tamaño del registro. Las pruebas A, B, C y D se completaron utilizando los siguientes valores:

- Prueba A: $M=128$
- Prueba B: $M=256$
- Prueba C: $M=512$
- Prueba D: $M=1024$

Posteriormente, se intentaron realizar las pruebas E y F, correspondientes a $M=2048$ y $M=4096$, respectivamente. Sin

embargo, el programa no logró completar el análisis debido a que, al incrementar el número de muestras, también aumenta la cantidad de memoria necesaria para almacenar y procesar estos datos, por lo que la asignación de memoria requerida para los arreglos de trabajo de la FFT agotó la memoria disponible para el intérprete de MicroPython. Por esta razón, las pruebas no pudieron finalizarse.

Figure 17. Fallo de asignación de memoria al ejecutar M = 2048/4096

Este resultado permite evidenciar una limitación práctica de la plataforma de procesamiento utilizada. Aunque el análisis teórico permite evaluar el comportamiento del piso de ruido para valores superiores de MM, la capacidad disponible en MicroPython limita el tamaño máximo de la FFT que puede procesarse directamente en el microcontrolador.

C. Resolución espectral y piso de ruido teórico (AN693)

Antes de comparar contra los datos medidos, se calculan los tres parámetros teóricos que predicen el piso de ruido ideal para un ADC de Nb = 12 bits, tomando como ejemplo completo la prueba A (Ns = M = 128, Fs = 2000 Hz):

$$\Delta f_{\text{res}} = \frac{F_s}{M}$$

$$\Delta f_{\text{res}} = \frac{2000 \text{ Hz}}{128} = 15,36 \text{ Hz}$$

$$G_{\text{FFT}} = 10 \log_{10} \left(\frac{M}{2} \right)$$

$$G_{\text{FFT}} = 10 \log_{10} \left(\frac{128}{2} \right) = 18,06 \text{ dB}$$

$$\text{SNR}_{\text{ideal}} = 6,02 \times N_b + 1,76$$

$$\text{SNR}_{\text{ideal}} = 6,02(12) + 1,76 = 74,00 \text{ dB}$$

$$L_{\text{NF,ideal}} = -(6,02 \times N_b + 1,76) - 10 \log_{10} \left(\frac{M}{2} \right)$$

$$2\text{NF}_{\text{ideal}} = -(74,00) - 18,06 = -92,06 \text{ dBFS/bin}$$

Adicionalmente, corrigiendo por el ancho de banda equivalente de ruido (ENBW) de la ventana Hann (1,5 bins), se obtiene el piso teórico ajustado, útil para comparar de forma más justa contra el piso realmente medido con ventaneo:

$$L_{\text{NF,win}} = L_{\text{NF,ideal}} + 10 \log_{10}(\text{ENBW})$$

$$L_{\text{NF,win}} = -92,06 + 10 \log_{10}(1,5) = -90,30 \text{ dBFS/bin}$$

El mismo procedimiento se repitió para M = 256, 512 y 1024 (pruebas B, C y D). Para M = 2048 y 4096 solo se reporta el

valor teórico, dado que la corrida experimental no pudo completarse (ver apartado anterior).

Se presenta continuación los resultados de la consola del ejercicio 128. Para ver el resto diríjese al **ANEXO D**

```
--- Spectral analysis ---
Window                : hann
Fundamental frequency : 250.200 Hz
Signal amplitude       : 0.926196 Vrms
Signal amplitude       : 1.309839 Vpeak
Signal amplitude       : 2.619679 Vpp
Signal level           : -2.01 dBFS
Integrated noise       : 0.001153 Vrms
Integrated noise       : -60.10 dBFS
Measured FFT noise floor: -69.45 dBFS/bin
Ideal quantization noise: 0.000233 Vrms
Ideal SNR (12 bit)     : 74.00 dB
Ideal FFT noise floor  : -92.06 dBFS/bin
Window ENBW           : 1.500 bins
Window-adjusted floor  : -90.30 dBFS/bin
SNR                    : 58.10 dB
THD                    : -51.70 dB (0.260 %)
THD+N                  : -50.80 dB (0.288 %)
SINAD                  : 50.80 dB
ENOB                   : 8.15 bits
SFDR                   : 58.87 dBc
Largest spur           : 609.846 Hz, -60.88 dBFS
Jitter-limited SNR est.: 42.73 dB
```

Figure 10. Resultado Consola – 128

M	Δf [Hz]	G_FFT [dB]	P.Teo [dBFS]	P.Med [dBFS]	Ruido [Vrms]
128	15,63	18,06	-92,06	-69,45	0,001153
256	7,81	21,07	-95,07	-68,91	0,003682
512	3,91	24,08	-98,08	-70,17	0,005027
1024	1,95	27,09	-101,09	-73,03	0,005448

A partir de los resultados se observa que, al aumentar el número de puntos de la FFT, la resolución espectral mejora. Para el caso de M=128, se obtiene una resolución de 15,63 Hz, mientras que para M=1024 esta disminuye hasta 1,95 Hz.

De igual manera, la ganancia de procesamiento aumenta de **18,06 dB** para M=128 hasta **27,09 dB** para M=1024. De acuerdo con el modelo teórico, esto produce una disminución del piso de ruido por bin a medida que aumenta el número de puntos de la FFT.

Experimentalmente, el piso de ruido pasa de **-69,45 dBFS/bin** para M=128 a **-73,03 dBFS/bin** para M=1024, lo que representa una disminución de aproximadamente **3,58 dB**.

D. Métricas dinámicas del ADC (AN693)

A partir del mismo análisis espectral se extrajeron las métricas dinámicas definidas por AN693 (SNR, THD, SINAD, ENOB y SFDR). Como ejemplo completo, se desarrolla el cálculo de ENOB para la prueba A (M = 128), a partir del SINAD reportado por consola (50,80 dB):

$$ENOB = \frac{(SINAD - 1,76)}{6,02}$$

$$ENOB = \frac{(50,80 - 1,76)}{6,02} = 8,15 \text{ bits}$$

Este valor coincide con el ENOB reportado directamente por la consola (8,15 bits), lo que valida la relación teórica AN693 frente a la implementación del script. El mismo cálculo se repite para $M = 256, 512$ y 1024 en la Tabla 3.

M	SNR [dB]	THD [dB]	THD [%]	SINAD [dB]	ENOB [bits]
128	58,10	-51,70	0,260	50,80	8,15
256	48,01	-55,45	0,169	47,29	7,56
512	45,30	-54,58	0,187	44,82	7,15
1024	44,60	-58,13	0,124	44,41	7,08

Tabla 3a. Métricas dinámicas del ADC (SNR, THD, SINAD, ENOB) — AN693, $M = 128$ a 1024

Adicionalmente, se analizaron el rango dinámico libre de espurios (SFDR), el componente espurio dominante y el límite de SNR asociado al jitter temporal.

M	SFDR [dBc]	Espurio mayor	Jitter σ_t [μs]	SNR jitter [dB]
128	58,87	609,8 Hz / -60,88 dBFS	4,650	42,73
256	56,65	148,5 Hz / -58,66 dBFS	3,421	45,40
512	59,06	164,1 Hz / -61,08 dBFS	2,729	47,36
1024	62,04	449,3 Hz / -64,06 dBFS	2,159	49,39

Tabla 3b. SFDR, espurio dominante y límite de SNR por jitter — AN693, $M = 128$ a 1024

E. Visualización de las señales en el dominio del tiempo y en frecuencia (MATLAB)

Para complementar el análisis numérico presentado en las Tablas 2 y 3, se graficaron las señales del dominio del tiempo y su espectro en frecuencia (FFT) en escala dBFS a partir de los archivos .csv generados por ADC_testing.py. Ver ANEXO E.

A modo de ejemplo se presenta a continuación el resultado obtenido para la prueba D ($N_s = M = 1024$).

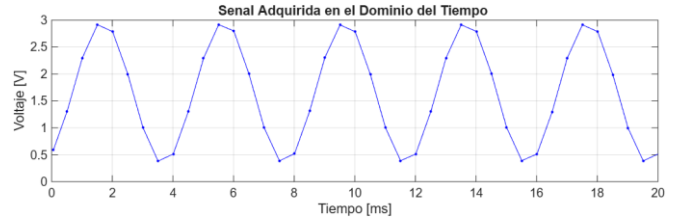


Figure 24. Señal adquirida en el dominio del tiempo ($N_s = M = 1024$)

la señal senoidal de 250 Hz adquirida por el ADC, con un offset aproximado de 1,65 V y una amplitud pico a pico cercana a 2,6 V, en concordancia con los parámetros configurados en el generador de funciones

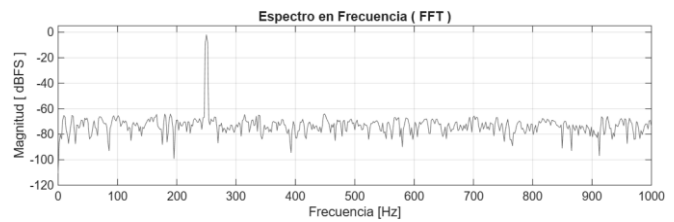


Figure 25. Espectro en frecuencia (FFT) en dBFS ($N_s = M = 1024$)

el espectro obtenido mediante la FFT muestra un lóbulo principal claramente definido en 250 Hz con una magnitud cercana a 0 dBFS, mientras que el piso de ruido se ubica alrededor de -70 dBFS, valor consistente con el piso medido reportado en la Tabla 2 para $M = 1024$.

F. Superposición de espectros FFT y efecto de M sobre el piso de ruido

Con el fin de comparar visualmente el efecto del número de puntos de la FFT (M) sobre el piso de ruido, se implementó el código superposición para superponer en una misma gráfica los espectros obtenidos en las pruebas A a D ($M = 128, 256, 512$ y 1024). Ver ANEXO F

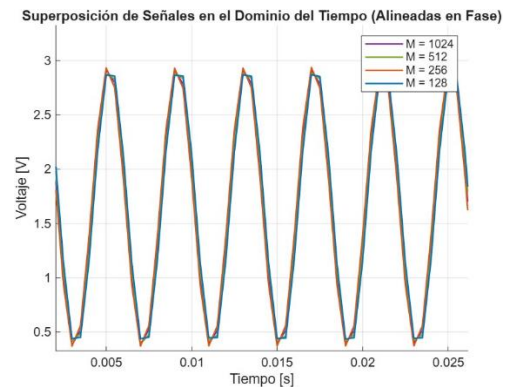


Figure 11. Superposición de Señales en el Tiempo

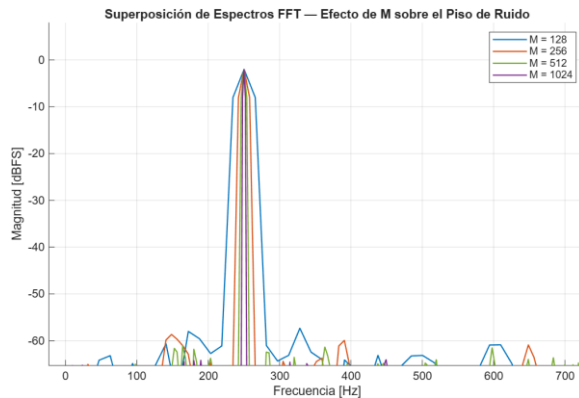


Figure 12. Superposición espectros

Se observa que, al aumentar M , el lóbulo principal en 250 Hz se hace progresivamente más angosto, consistente con la mejora en la resolución espectral.

Asimismo, el piso de ruido de fondo (fuera del lóbulo principal) desciende de forma visible entre $M = 128$ y $M = 1024$.

V. ANÁLISIS Y DISCUSIÓN DE RESULTADOS

A. Verificación del dominio del tiempo y de la frecuencia de muestreo

El análisis temporal confirmó que el sistema de adquisición operó de forma consistente con lo establecido en la guía de laboratorio [7]. La frecuencia de muestreo medida por software (2000,352 Hz) y por osciloscopio (2000 Hz) coincidieron con el valor objetivo de 2000 Hz dentro de un error inferior al 0,02 %, mientras que el jitter temporal (2,509 μ s, Tabla 2 de la Sección IV del informe original) resultó tres órdenes de magnitud menor que el período de muestreo nominal (500 μ s), lo que confirma una base de tiempos estable en el microcontrolador RP2350 [2]. La verificación cruzada mediante los flancos de la señal digital de GP15 sobre el canal 2 del osciloscopio (Figuras 4 y 7) corroboró de forma visual las $N_s = 8$ muestras por período calculadas.

B. Comportamiento de las señales en el dominio del tiempo

Las capturas del dominio del tiempo (Figuras 24 y 11) muestran una señal senoidal de 250 Hz con offset de 1,65 V y amplitud pico a pico cercana a 2,6 V, en concordancia con los parámetros configurados en el generador de funciones para la Fase 2 [7]. En la superposición de las cuatro pruebas (Figura 11), las cuatro trazas de $M = 128, 256, 512$ y 1024 se superponen prácticamente de forma exacta en amplitud y fase, ya que corresponden a la misma señal analógica de entrada muestreada bajo idénticas condiciones de F_s y offset; la única diferencia entre ellas es la longitud del registro temporal capturado (N_s), que en la ventana de 20 ms mostrada resulta imperceptible a simple vista.

C. Comportamiento espectral: resolución, lóbulo principal y piso de ruido

En el dominio de la frecuencia (Figuras 12 y 25), el efecto más evidente al incrementar M es el estrechamiento progresivo del lóbulo principal alrededor de 250 Hz, consecuencia directa de la mejora en la resolución espectral $\Delta f_{res} = \frac{F_s}{M}$ de 15,63 Hz para $M = 128$ a 1,95 Hz para $M = 1024$ (y, teóricamente, hasta 0,49 Hz para $M = 4096$). Alrededor del lóbulo principal se observan lóbulos secundarios propios de la fuga espectral remanente bajo ventaneo Hann, cuya amplitud decrece rápidamente en frecuencia, consistente con las propiedades de dicha ventana [6], [8].

El piso de ruido de fondo, fuera del lóbulo principal, desciende de forma visible entre $M = 128$ y $M = 1024$, pasando de $-69,45$ dBFS/bin a $-73,03$ dBFS/bin (reducción experimental de 3,58 dB). Sin embargo, al examinar la progresión completa de la Tabla 2 se observa que dicho descenso no es estrictamente monótono: entre $M = 128$ y $M = 256$ el piso medido empeora levemente (de $-69,45$ a $-68,91$ dBFS/bin) antes de mejorar en $M = 512$ y $M = 1024$. Este comportamiento no monótono, ausente en la predicción teórica —que sí es estrictamente decreciente en pasos de 3,01 dB por bin al duplicar M , sugiere que a longitudes de registro cortas el piso de ruido medido está dominado por fuentes de ruido de banda ancha del sistema de adquisición (ruido térmico del front-end analógico, acoplamientos electromagnéticos del montaje de la Figura 8) más que por el ruido de cuantización puro, y que su contribución relativa varía de una corrida a otra.

D. Ganancia de procesamiento teórica frente a la real

De acuerdo con AN693, la ganancia de procesamiento G_{FFT} aumentó de 18,06 dB ($M = 128$) a 27,09 dB ($M = 1024$), y llegaría a 33,11 dB para $M = 4096$ de haberse podido completar la prueba. Esta ganancia predice teóricamente una reducción del piso de ruido de 9,03 dB entre $M = 128$ y $M = 1024$ (3 duplicaciones de 3,01 dB cada una). Experimentalmente, la reducción medida fue de solo 3,58 dB, es decir, aproximadamente el 40 % de la mejora prevista. Esta brecha confirma que el piso de ruido real del sistema está condicionado por fuentes adicionales de error ruido térmico, ruido $\frac{1}{f}$, acoplamientos del cableado y resolución efectiva del ADC que no se dividen entre los bins de la FFT de la misma manera en que lo hace el ruido de cuantización blanco e incorrelado supuesto por el modelo ideal [6].

E. Brecha entre el piso teórico y el piso medido, y degradación del ENOB

Al comparar los valores experimentales con el piso de ruido teórico se observa una diferencia considerable para todos los valores de M : para $M = 128$, el piso medido ($-69,45$ dBFS/bin) se encuentra 22,61 dB por encima del piso ideal ($-92,06$ dBFS/bin); para $M = 1024$, la diferencia crece a 28,06 dB ($-73,03$ dBFS/bin frente a $-101,09$ dBFS/bin). Esta diferencia muestra que el desempeño real del ADC está condicionado por fuentes de ruido adicionales al ruido de cuantización ideal descrito en la Sección III.F.

Esta misma limitación se refleja en el número efectivo de bits (ENOB), cuyos valores experimentales (7,08 a 8,15 bits) resultan sensiblemente inferiores a los 12 bits nominales del

ADC. De forma contraintuitiva, el ENOB medido disminuye al aumentar M (8,15 bits en $M = 128$ frente a 7,08 bits en $M = 1024$) en lugar de mejorar junto con la ganancia de procesamiento. La explicación más plausible es que, al capturar ventanas de tiempo más largas para lograr un M mayor, se incorpora más energía de ruido de baja frecuencia (deriva térmica, ruido $1/f$ y fluctuaciones lentas de la tensión de referencia) dentro de la misma medición de SINAD, degradando la relación señal-ruido efectiva a pesar de la mayor resolución espectral obtenida [6]. Este resultado ilustra que la resolución nominal de un convertidor no equivale necesariamente a su resolución efectiva en condiciones reales de operación.

F. Influencia del jitter de muestreo

El límite de SNR asociado al jitter, calculado mediante la expresión de la Sección III.N, varió entre 42,73 dB ($M = 128$) y 49,39 dB ($M = 1024$), mientras que el SNR medido estuvo entre 44,60 dB y 58,10 dB. Para $M = 128$ y $M = 256$, el SNR medido superó el límite impuesto por el jitter, mientras que para $M = 512$ y $M = 1024$ ocurrió lo contrario. Esta inconsistencia indica que el jitter de muestreo no puede considerarse la única causa de la degradación del SNR.

G. Limitaciones prácticas de la plataforma

La imposibilidad de completar las pruebas E y F ($M = 2048$ y 4096, Sección II.B) evidenció una limitación práctica de la plataforma de procesamiento: la asignación de memoria requerida por los arreglos de trabajo de la FFT agotó la memoria disponible para el intérprete de MicroPython en el RP2350. Aunque el análisis teórico de la Sección IV permite predecir el comportamiento del piso de ruido y la ganancia de procesamiento para valores superiores de M , la capacidad de memoria RAM disponible en la plataforma limita el tamaño máximo de la FFT que puede procesarse directamente en el microcontrolador, restricción que debería considerarse en el diseño de sistemas embebidos de adquisición y análisis espectral en tiempo real.

VI. CONCLUSIONES

A. Se determinó experimentalmente la frecuencia y la estabilidad de muestreo (F_s) del sistema empleando osciloscopio e instrumentación digital, obteniéndose una frecuencia medida por software de 2000,352 Hz y por osciloscopio de 2000 Hz frente a un valor objetivo de 2000 Hz (error de 0,0176 %), con un jitter temporal de 2,509 μs , lo que confirma una base de tiempos estable y adecuada para la caracterización dinámica del convertidor A/D del RP2350.

B. Se evaluó el desempeño dinámico del convertidor A/D cuantificando sus parámetros clave según AN693, obteniéndose valores de SNR entre 44,60 dB y 58,10 dB, THD entre 0,124 % y 0,260 %, SINAD entre 44,41 dB y 50,80 dB, ENOB entre 7,08 y 8,15 bits, SFDR entre 56,65 dBc y 62,04 dBc, y un piso de ruido medido entre $-68,91$ dBFS/bin y $-73,03$ dBFS/bin, evidenciando que el desempeño real del ADC se ubica por debajo de sus 12 bits nominales de resolución.

C. Se demostró cuantitativa y visualmente la ganancia de procesamiento (processing gain) de la FFT y el descenso teórico del piso de ruido por bin al aumentar el número de puntos M , verificándose un incremento de la ganancia de 18,06 dB a 27,09 dB entre $M = 128$ y $M = 1024$ (con una proyección teórica de 33,11 dB para $M = 4096$), así como un estrechamiento progresivo del lóbulo principal y una reducción experimental parcial del piso de ruido de 3,58 dB, inferior a la reducción de 9,03 dB prevista por el modelo ideal.

D. Se contrastaron los resultados experimentales frente a los límites teóricos del ADC ideal de 12 bits establecidos por la nota de aplicación AN693 de Microchip, encontrándose diferencias de entre 22,6 dB y 28,1 dB entre el piso de ruido medido y el piso ideal de cuantización, así como una degradación contraintuitiva del ENOB al aumentar M , lo que evidenció la presencia de fuentes de ruido adicionales (térmico, $1/f$, jitter y acoplamientos del montaje) no contempladas en el modelo ideal de cuantización.

E. Se identificaron las limitaciones prácticas de la plataforma de procesamiento al aumentar el tamaño de la FFT, comprobándose que la memoria disponible para el intérprete de MicroPython en la Raspberry Pi Pico 2W impidió completar el análisis espectral para $M = 2048$ y $M = 4096$, lo que permitió cumplir el objetivo general de la práctica al caracterizar de forma integral tanto el desempeño teórico-experimental del convertidor como las restricciones tecnológicas reales de la plataforma empleada.

VII. REFERENCIA

- [1] Raspberry Pi Ltd., "Raspberry Pi Pico 2 Datasheet," Raspberry Pi Ltd., Cambridge, U.K., 2025.
- [2] Raspberry Pi Ltd., "RP2350 Datasheet," Raspberry Pi Ltd., Cambridge, U.K., 2025.
- [3] Raspberry Pi Ltd., "Raspberry Pi Pico-series Python SDK," Raspberry Pi Ltd., Cambridge, U.K.
- [4] MicroPython, "Quick reference for the RP2," MicroPython Documentation. [En línea].
- [5] Raspberry Pi Ltd., "Raspberry Pi Documentation: Microcontroller chips – RP2350," Raspberry Pi Ltd., Cambridge, U.K.
- [6] S. Bowling, "Understanding A/D converter performance specifications," Microchip Technology Inc., Chandler, AZ, USA, Appl. Note AN693, 2002.
- [7] J. de J. Rúgeles Uribe, "Análisis de parámetros de un conversor ADC," Guía de laboratorio, Comunicaciones Digitales, Programa de Ingeniería en Telecomunicaciones, Universidad Militar Nueva Granada, Bogotá, Colombia, 2026.
- [8] A. V. Oppenheim y R. W. Schaffer, Discrete-Time Signal Processing, 3.^a ed. Upper Saddle River, NJ, USA: Prentice-Hall, 2010.

VIII. ANEXOS

A. ANEXO A ADC_testing.py

```
from machine import ADC, Pin
import utime
import array
```

```
import math
import cmath
import gc
ADC_PIN = 27
SAMPLE_MARKER_PIN = 15
ADC_BITS = 12      # N = 12 bits de resolución del ADC
VREF = 3.3
N_SAMPLES = 512    # N_s = Número de muestras capturadas
(modificable por el estudiante)
N_FFT = 1024       # M = Puntos de la FFT (N_FFT >= N_s)
FS_TARGET = 2000
DT_US = int(round(1_000_000 / FS_TARGET))
DEFAULT_INPUT_FREQ_HZ = 250.0
DEFAULT_WINDOW = "hann"
NUM_HARMONICS = 5
MAIN_LOBE_HALF_WIDTH = 4
GUARD_HALF_WIDTH = 8
HARMONIC_SEARCH_HALF_WIDTH = 2
adc = ADC(Pin(ADC_PIN))
sample_marker = Pin(SAMPLE_MARKER_PIN, Pin.OUT)
sample_marker.value(0)
def mean(values):
    return sum(values) / len(values)
def median(values):
    if not values:
        return 0.0
    ordered = sorted(values)
    n = len(ordered)
    m = n // 2
    if n % 2:
        return ordered[m]
    return 0.5 * (ordered[m - 1] + ordered[m])
def std_population(values, avg=None):
    if not values:
        return 0.0
    if avg is None:
        avg = mean(values)
    return math.sqrt(sum((x - avg) ** 2 for x in values) /
len(values))
def db10_ratio(num, den):
    if num <= 0 or den <= 0:
        return -200.0
    return 10.0 * math.log10(num / den)
def db20_ratio(num, den):
    if num <= 0 or den <= 0:
        return -200.0
    return 20.0 * math.log10(num / den)
def alias_to_first_nyquist_zone(freq, fs):
    if fs <= 0:
        return 0.0
    f = abs(freq) % fs
    if f > fs / 2.0:
        f = fs - f
    return f
def bin_region(center, half_width, max_bin):
    start = max(1, center - half_width)
    stop = min(max_bin, center + half_width)
    return set(range(start, stop + 1))
```

```
def window_enbw_bins(window):
    sw = sum(window)
    sw2 = sum(w * w for w in window)
    if sw <= 0:
        return 1.0
    return len(window) * sw2 / (sw * sw)

def safe_label_frequency(freq):
    text = "{:.3f}".format(freq)
    while text.endswith("0"):
        text = text[:-1]
    if text.endswith("."):
        text += "0"
    return text.replace(".", "p") + "Hz"

def get_run_configuration():
    print()
    print("--- Test configuration ---")
    txt = input(
        "Input frequency from generator [Hz] "
        "(Enter = {:.0f}): ".format(DEFAULT_INPUT_FREQ_HZ)
    ).strip()
    if txt:
        try:
            input_freq_hz = float(txt)
        except ValueError:
            print("Invalid value. Using default frequency.")
            input_freq_hz = DEFAULT_INPUT_FREQ_HZ
    else:
        input_freq_hz = DEFAULT_INPUT_FREQ_HZ
    txt = input(
        "Window hann/rectangular "
        "(Enter = {}): ".format(DEFAULT_WINDOW)
    ).strip().lower()
    if not txt:
        window_type = DEFAULT_WINDOW
    elif txt in ("hann", "rectangular"):
        window_type = txt
    else:
        print("Invalid window. Using default window.")
        window_type = DEFAULT_WINDOW
    return input_freq_hz, window_type

def acquire_data():
    samples = array.array("H", [0] * N_SAMPLES)
    times_us = array.array("I", [0] * N_SAMPLES)
    gc.collect()
    gc.disable()
    start = utime.ticks_us()
    for i in range(N_SAMPLES):
        target = utime.ticks_add(start, i * DT_US)
        while utime.ticks_diff(target, utime.ticks_us()) > 0:
            pass
        sample_marker.value(1)
        t_sample = utime.ticks_us()
        times_us[i] = utime.ticks_diff(t_sample, start)
        samples[i] = adc.read_u16()
        sample_marker.value(0)
```

```

sample_marker.value(0)
gc.enable()
elapsed_us = times_us[-1] - times_us[0]
fs_real = (N_SAMPLES - 1) * 1_000_000.0 / elapsed_us if
elapsed_us > 0 else 0.0

intervals_us = [times_us[i] - times_us[i - 1] for i in range(1,
N_SAMPLES)]
mean_dt_us = mean(intervals_us)
jitter_us = std_population(intervals_us, mean_dt_us)
fs_error_pct = (
    100.0 * (fs_real - FS_TARGET) / FS_TARGET if
FS_TARGET > 0 else 0.0
)
print()
print("--- Sampling ---")
print("Target Fs      : {:.3f} Hz".format(FS_TARGET))
print("Measured Fs    : {:.3f} Hz".format(fs_real))
print("Fs error       : {:.4f} %".format(fs_error_pct))
print("Mean Ts        : {:.3f} us".format(mean_dt_us))
print("Timing jitter (std) : {:.3f} us".format(jitter_us))
return samples, times_us, fs_real, jitter_us

def fft_manual(x, n_fft):
    if n_fft < len(x):
        raise ValueError("N_FFT must be >= number of acquired
samples")
    if n_fft & (n_fft - 1):
        raise ValueError("N_FFT must be a power of two")

    def bit_reversal(n, log_n):
        rev = 0
        for i in range(log_n):
            if (n >> i) & 1:
                rev |= 1 << (log_n - 1 - i)
        return rev
    X = [complex(v, 0.0) for v in x]
    X.extend([0j] * (n_fft - len(x)))

    log_n = int(math.log2(n_fft))

    for i in range(n_fft):
        j = bit_reversal(i, log_n)
        if j > i:
            X[i], X[j] = X[j], X[i]
    for stage in range(log_n):
        m = 2 ** (stage + 1)
        half_m = m // 2
        w_m = cmath.exp(-2j * math.pi / m)
        for k in range(0, n_fft, m):
            w = 1.0 + 0j
            for j in range(half_m):
                t = w * X[k + j + half_m]
                u = X[k + j]
                X[k + j] = u + t
                X[k + j + half_m] = u - t
                w *= w_m
            return X

def analyze_fft(

```

```

fft_result,
fs_real,
window,
input_freq_hz,
window_type,
jitter_us,
n_fft=N_FFT
):
    nyquist_bin = n_fft // 2
    max_bin = nyquist_bin
    num_bins = max_bin + 1
    # Preasignación de tamaño fijo para evitar fragmentación
    raw_power = [0.0] * num_bins
    for k in range(num_bins):
        raw_power[k] = abs(fft_result[k]) ** 2
    # Búsqueda de fundamental (excluyendo DC)
    fundamental_bin = max(range(1, num_bins), key=lambda k:
raw_power[k])
    delta = 0.0
    if 1 <= fundamental_bin < max_bin:
        a = raw_power[fundamental_bin - 1]
        b = raw_power[fundamental_bin]
        c = raw_power[fundamental_bin + 1]
        denom = a - 2.0 * b + c
        if abs(denom) > 1e-30:
            delta = 0.5 * (a - c) / denom
            if delta > 0.5:
                delta = 0.5
            elif delta < -0.5:
                delta = -0.5
        fundamental_freq = (fundamental_bin + delta) * fs_real /
n_fft
    window_sum = sum(window)
    full_scale_peak = VREF / 2.0
    full_scale_rms = full_scale_peak / math.sqrt(2.0)
    frequencies = [0.0] * num_bins
    magnitudes_vpeak = [0.0] * num_bins
    magnitudes_dbfs = [0.0] * num_bins
    for k in range(num_bins):
        frequencies[k] = k * fs_real / n_fft
        mag = (
            abs(fft_result[k]) / window_sum
            if (k == 0 or k == nyquist_bin)
            else 2.0 * abs(fft_result[k]) / window_sum
        )
        magnitudes_vpeak[k] = mag
        magnitudes_dbfs[k] = db20_ratio(mag, full_scale_peak)
    window_power_sum = sum(w * w for w in window)
    spectral_power = [0.0] * num_bins
    for k in range(num_bins):
        factor = 1.0 if (k == 0 or k == nyquist_bin) else 2.0
        spectral_power[k] = factor * raw_power[k] / (n_fft *
window_power_sum)
    signal_bins = bin_region(fundamental_bin,
MAIN_LOBE_HALF_WIDTH, max_bin)
    signal_guard = bin_region(fundamental_bin,
GUARD_HALF_WIDTH, max_bin)
    harmonic_bins = set()

```

```

harmonic_guard = set()
harmonic_info = []
for h in range(2, NUM_HARMONICS + 1):
    expected_freq = alias_to_first_nyquist_zone(h *
fundamental_freq, fs_real)
    if expected_freq <= 0.0:
        continue
    expected_bin = int(round(expected_freq * n_fft / fs_real))
    expected_bin = max(1, min(max_bin, expected_bin))
    search_start = max(1, expected_bin -
HARMONIC_SEARCH_HALF_WIDTH)
    search_stop = min(max_bin, expected_bin +
HARMONIC_SEARCH_HALF_WIDTH)
    harmonic_bin = max(range(search_start, search_stop + 1),
key=lambda k: raw_power[k])
    this_power_region = bin_region(harmonic_bin,
MAIN_LOBE_HALF_WIDTH, max_bin)
    this_guard_region = bin_region(harmonic_bin,
GUARD_HALF_WIDTH, max_bin)
    this_power_region -= signal_bins
    this_guard_region -= signal_guard
    this_power_region -= harmonic_bins
    this_guard_region -= harmonic_guard
    if this_power_region:
        harmonic_bins |= this_power_region
        harmonic_guard |= this_guard_region
    harmonic_info.append((h, harmonic_bin, harmonic_bin
* fs_real / n_fft))
    all_positive_bins = set(range(1, num_bins))
    noise_bins = all_positive_bins - signal_guard -
harmonic_guard
    signal_power = sum(spectral_power[k] for k in signal_bins)
    harmonic_power = sum(spectral_power[k] for k in
harmonic_bins)
    noise_power = sum(spectral_power[k] for k in noise_bins)
    signal_rms = math.sqrt(signal_power) if signal_power > 0
else 0.0
    signal_peak = math.sqrt(2.0) * signal_rms
    signal_vpp = 2.0 * signal_peak
    noise_rms = math.sqrt(noise_power) if noise_power > 0 else
0.0
    snr_db = db10_ratio(signal_power, noise_power)
    thd_db = db10_ratio(harmonic_power, signal_power)
    thd_percent = (
        100.0 * math.sqrt(harmonic_power / signal_power)
        if signal_power > 0 and harmonic_power > 0
        else 0.0
    )
    thdn_power = harmonic_power + noise_power
    thdn_db = db10_ratio(thdn_power, signal_power)
    thdn_percent = (
        100.0 * math.sqrt(thdn_power / signal_power)
        if signal_power > 0 and thdn_power > 0
        else 0.0
    )
    sinad_db = db10_ratio(signal_power, thdn_power)
    enob = (sinad_db - 1.76) / 6.02
    signal_dbfs = db20_ratio(signal_rms, full_scale_rms)

```

```

integrated_noise_dbfs = db20_ratio(noise_rms,
full_scale_rms)
noise_bin_dbfs = [magnitudes_dbfs[k] for k in noise_bins]
measured_noise_floor_dbfs = median(noise_bin_dbfs) if
noise_bin_dbfs else -200.0
ideal_lsb = VREF / (2 ** ADC_BITS)
ideal_quant_noise_rms = ideal_lsb / math.sqrt(12.0)
ideal_snr_db = 6.02 * ADC_BITS + 1.76
independent_bins = N_SAMPLES / 2.0
ideal_fft_noise_floor_dbfs = (
    -ideal_snr_db - 10.0 * math.log10(independent_bins)
    if independent_bins > 0
    else -200.0
)
enbw_bins = window_enbw_bins(window)
window_adjusted_noise_floor_dbfs = (
    ideal_fft_noise_floor_dbfs + 10.0 *
math.log10(enbw_bins)
)
spur_candidates = all_positive_bins - signal_guard
if spur_candidates:
    largest_spur_bin = max(spur_candidates, key=lambda k:
magnitudes_vpeak[k])
    largest_spur_freq = frequencies[largest_spur_bin]
    largest_spur_dbfs = magnitudes_dbfs[largest_spur_bin]
    fundamental_peak_dbfs =
magnitudes_dbfs[fundamental_bin]
    sfdr_db = fundamental_peak_dbfs - largest_spur_dbfs
else:
    largest_spur_bin = 0
    largest_spur_freq = 0.0
    largest_spur_dbfs = -200.0
    sfdr_db = 200.0
    expected_alias_hz =
alias_to_first_nyquist_zone(input_freq_hz, fs_real)
    alias_error_hz = fundamental_freq - expected_alias_hz
    samples_per_period = fs_real / input_freq_hz if
input_freq_hz > 0 else 0.0
    nyquist_hz = fs_real / 2.0
    jitter_s = jitter_us * 1e-6
    if input_freq_hz > 0 and jitter_s > 0:
        snr_jitter_db = -20.0 * math.log10(2.0 * math.pi *
input_freq_hz * jitter_s)
    else:
        snr_jitter_db = 200.0
    bin_types = ["guard"] * num_bins
    bin_types[0] = "dc"
    for k in signal_bins:
        bin_types[k] = "signal"
    for k in harmonic_bins:
        bin_types[k] = "harmonic"
    for k in noise_bins:
        bin_types[k] = "noise"
    print()
    print("--- Frequency interpretation ---")
    print("Declared input frequency : {:.3f}
Hz".format(input_freq_hz))

```

```

print("Measured Nyquist freq. : {:.3f}
Hz".format(nyquist_hz))
print("Expected alias frequency : {:.3f}
Hz".format(expected_alias_hz))
print("FFT fundamental : {:.3f}
Hz".format(fundamental_freq))
print("FFT - expected alias : {:.3f}
Hz".format(alias_error_hz))
print("Samples per input period :
{:.3f}".format(samples_per_period))
if input_freq_hz > nyquist_hz:
    print("WARNING: input is above Nyquist. The FFT
displays its alias.")
print()
print("--- Spectral analysis ---")
print("Window : {}".format(window_type))
print("Fundamental frequency : {:.3f}
Hz".format(fundamental_freq))
print("Signal amplitude : {:.6f}
Vrms".format(signal_rms))
print("Signal amplitude : {:.6f}
Vpeak".format(signal_peak))
print("Signal amplitude : {:.6f}
Vpp".format(signal_vpp))
print("Signal level : {:.2f} dBFS".format(signal_dbfs))
print("Integrated noise : {:.6f} Vrms".format(noise_rms))
print("Integrated noise : {:.2f}
dBFS".format(integrated_noise_dbfs))
print("Measured FFT noise floor: {:.2f}
dBFS/bin".format(measured_noise_floor_dbfs))
print("Ideal quantization noise: {:.6f}
Vrms".format(ideal_quant_noise_rms))
print("Ideal SNR ({} bit) : {:.2f} dB".format(ADC_BITS,
ideal_snr_db))
print("Ideal FFT noise floor : {:.2f}
dBFS/bin".format(ideal_fft_noise_floor_dbfs))
print("Window ENBW : {:.3f}
bins".format(enbw_bins))
print("Window-adjusted floor : {:.2f}
dBFS/bin".format(window_adjusted_noise_floor_dbfs))
print("SNR : {:.2f} dB".format(snr_db))
print("THD : {:.2f} dB ({:.3f} %)".format(thd_db,
thd_percent))
print("THD+N : {:.2f} dB ({:.3f}
%)".format(thdn_db, thdn_percent))
print("SINAD : {:.2f} dB".format(sinad_db))
print("ENOB : {:.2f} bits".format(enob))
print("SFDR : {:.2f} dBc".format(sfdr_db))
print("Largest spur : {:.3f} Hz, {:.2f}
dBFS".format(largest_spur_freq, largest_spur_dbfs))
print("Jitter-limited SNR est. : {:.2f}
dB".format(snr_jitter_db))
return {
    "frequencies": frequencies,
    "magnitudes_vpeak": magnitudes_vpeak,
    "magnitudes_dbfs": magnitudes_dbfs,
    "bin_types": bin_types,
    "input_freq_hz": input_freq_hz,
    "expected_alias_hz": expected_alias_hz,
    "fundamental_freq": fundamental_freq,
    "alias_error_hz": alias_error_hz,
    "samples_per_period": samples_per_period,
    "nyquist_hz": nyquist_hz,
    "signal_rms": signal_rms,
    "signal_peak": signal_peak,
    "signal_vpp": signal_vpp,
    "signal_dbfs": signal_dbfs,
    "noise_rms": noise_rms,
    "integrated_noise_dbfs": integrated_noise_dbfs,
    "measured_noise_floor_dbfs":
measured_noise_floor_dbfs,
    "ideal_quant_noise_rms": ideal_quant_noise_rms,
    "ideal_snr_db": ideal_snr_db,
    "ideal_fft_noise_floor_dbfs": ideal_fft_noise_floor_dbfs,
    "window_enbw_bins": enbw_bins,
    "window_adjusted_noise_floor_dbfs":
window_adjusted_noise_floor_dbfs,
    "snr_db": snr_db,
    "thd_db": thd_db,
    "thd_percent": thd_percent,
    "thdn_db": thdn_db,
    "thdn_percent": thdn_percent,
    "sinad_db": sinad_db,
    "enob": enob,
    "sfdr_db": sfdr_db,
    "largest_spur_freq": largest_spur_freq,
    "largest_spur_dbfs": largest_spur_dbfs,
    "snr_jitter_db": snr_jitter_db,
}
def make_filenames(input_freq_hz, window_type):
    label = safe_label_frequency(input_freq_hz) + "_" +
window_type
    return (
        "adc_samples_" + label + ".csv",
        "adc_fft_" + label + ".csv",
        "adc_summary_" + label + ".csv",
    )
def save_samples_csv(filename, samples, times_us,
vref=VREF):
    with open(filename, "w") as f:
        f.write("sample,time_s,raw_u16,code12,voltage_V\n")
        for i in range(len(samples)):
            code12 = samples[i] >> 4
            v = (samples[i] / 65535.0) * vref
            f.write("{} {:.9f}, {}, {}, {:.6f}\n".format(
                i, times_us[i] / 1_000_000.0, samples[i], code12, v
            ))
def save_fft_csv(filename, results):
    with open(filename, "w") as f:
        f.write("frequency_Hz,magnitude_Vpeak,magnitude_dBFS,bi
n_type\n")
        for freq, mag, mag_dbfs, b_type in zip(
            results["frequencies"],
            results["magnitudes_vpeak"],
            results["magnitudes_dbfs"],
            results["bin_types"],

```

```

):
    f.write("{:.6f},{:.8f},{:.4f},{ }\n".format(freq, mag,
mag_dbfs, b_type)
def save_summary_csv(filename, results, fs_real, jitter_us,
window_type):
    with open(filename, "w") as f:
        f.write(
            "input_frequency_Hz,measured_Fs_Hz,nyquist_Hz>window,N
_SAMPLES,N_FFT,"
            "physical_resolution_Hz,FFT_spacing_Hz,expected_alias_Hz,"
            "FFT_fundamental_Hz,frequency_error_Hz,samples_per_peri
od,"
            "timing_jitter_us,signal_dbFS,integrated_noise_Vrms,"
            "integrated_noise_dbFS,measured_noise_floor_dbFS_per_bi
n,"
            "ideal_SNR_dB,ideal_noise_floor_dbFS_per_bin,SNR_dB,T
HD_dB,"
            "THD_percent,THDN_dB,THDN_percent,SINAD_dB,ENOB
_bits,SFDR_dBc,"
            "largest_spur_Hz,largest_spur_dbFS,jitter_limited_SNR_dB\
n"
        )
        f.write(
            "{:.6f},{:.6f},{:.6f},{ },{ },{ },{:.6f},{:.6f},{:.6f},{:.6f},"
            "{:.6f},{:.6f},{:.6f},{:.6f},{:.9f},{:.6f},{:.6f},{:.6f},"
            "{:.6f},{:.6f},{:.6f},{:.6f},{:.6f},{:.6f},{:.6f},{:.6f},"
            "{:.6f},{:.6f},{:.6f},{:.6f}\n".format(
                results["input_freq_hz"],
                fs_real,
                results["nyquist_hz"],
                window_type,
                N_SAMPLES,
                N_FFT,
                fs_real / N_SAMPLES,
                fs_real / N_FFT,
                results["expected_alias_hz"],
                results["fundamental_freq"],
                results["alias_error_hz"],
                results["samples_per_period"],
                jitter_us,
                results["signal_dbfs"],
                results["noise_rms"],
                results["integrated_noise_dbfs"],
                results["measured_noise_floor_dbfs"],
                results["ideal_snr_db"],
                results["window_adjusted_noise_floor_dbfs"],
                results["snr_db"],
                results["thd_db"],
                results["thd_percent"],
                results["thdn_db"],
                results["thdn_percent"],
                results["sinad_db"],
                results["enob"],
                results["sfdr_db"],
                results["largest_spur_freq"],
                results["largest_spur_dbfs"],
                results["snr_jitter_db"],
            )

```

```

)
)
def main():
    print("=====
=====")
    print("ADC dynamic characterization - FFT")
    print("Raspberry Pi Pico 2 W - RP2350")
    print("=====
=====")
    print("ADC input          : GP{}".format(ADC_PIN))
    print("Sampling marker          :
GP{}".format(SAMPLE_MARKER_PIN))
    print("N_SAMPLES (Ns)          : {}".format(N_SAMPLES))
    print("N_FFT (M)              : {}".format(N_FFT))
    print("Target Fs              : {} Hz".format(FS_TARGET))
    print("Approx. physical df          : {:.4f}
Hz".format(FS_TARGET / N_SAMPLES))
    print("Approx. FFT spacing          : {:.4f}
Hz".format(FS_TARGET / N_FFT))
    input_freq_hz, window_type = get_run_configuration()
    samples_file, fft_file, summary_file =
make_filenames(input_freq_hz, window_type)
    samples, times_us, fs_real, jitter_us = acquire_data()
    # Conversion in-situ a voltajes para ahorrar memoria
    n_s = len(samples)
    voltages = [(samples[i] / 65535.0) * VREF for i in
range(n_s)]
    avg_dc = mean(voltages)
    print("DC offset          : {:.6f} V".format(avg_dc))
    # Restar continua in-situ
    for i in range(n_s):
        voltages[i] -= avg_dc
    # Generacion de ventana y ventaneo in-situ
    if window_type == "hann":
        window = [0.5 * (1.0 - math.cos(2.0 * math.pi * i / n_s))
for i in range(n_s)]
    else:
        window = [1.0] * n_s
    for i in range(n_s):
        voltages[i] *= window[i]
    gc.collect()
    fft_result = fft_manual(voltages, N_FFT)
    del voltages
    gc.collect()
    results = analyze_fft(
        fft_result,
        fs_real,
        window,
        input_freq_hz,
        window_type,
        jitter_us,
        N_FFT,
    )
    del fft_result
    gc.collect()
    save_samples_csv(samples_file, samples, times_us, VREF)
    del samples, times_us
    gc.collect()

```

```
save_fft_csv(fft_file, results)
save_summary_csv(summary_file, results, fs_real, jitter_us,
window_type)
print()
print("--- Output files ---")
print("Time-domain data      :", samples_file)
print("Frequency-domain data  :", fft_file)
print("Summary                 :", summary_file)
print("Analysis completed successfully.")
if __name__ == "__main__":
    main()
```

B. ANEXO B. Mediciones de los instantes de muestreo

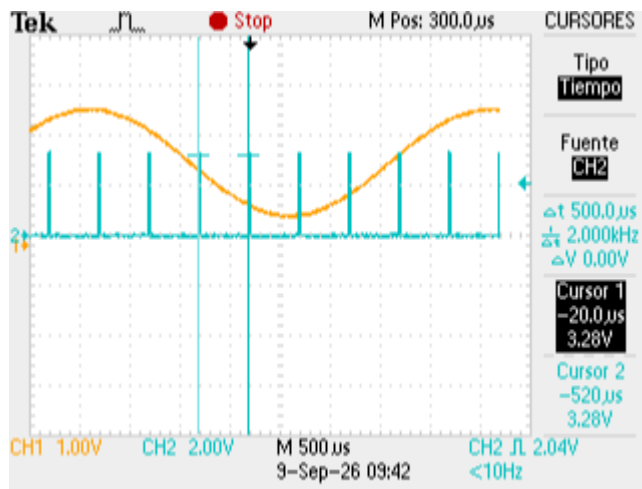


Figure 13. Periodo Muestreo 1

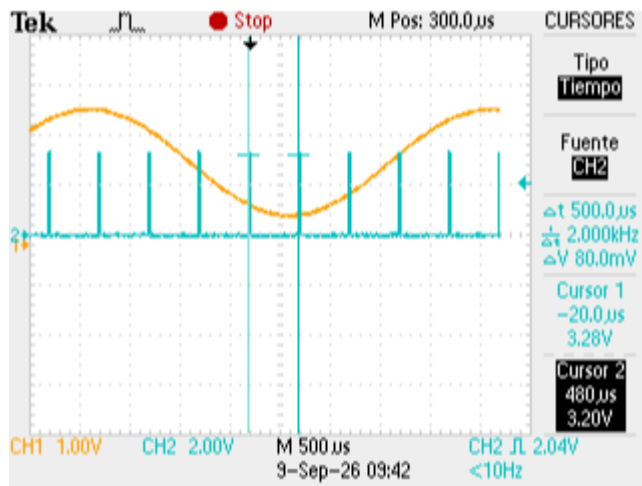


Figure 14. Periodo Muestreo 2

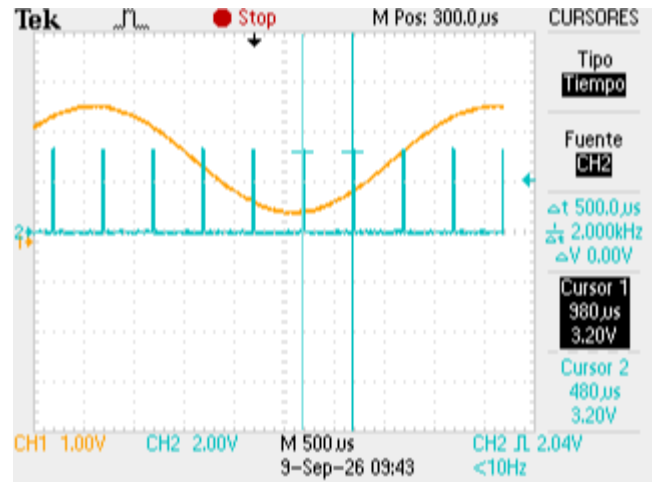


Figure 15. Periodo Muestreo 3

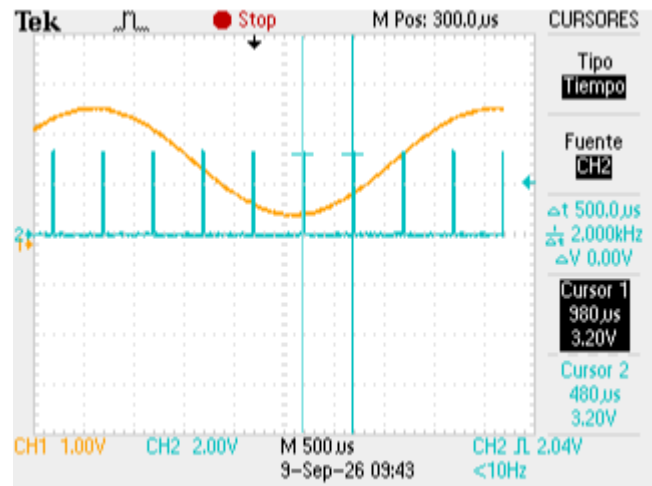


Figure 16. Periodo Muestreo 4

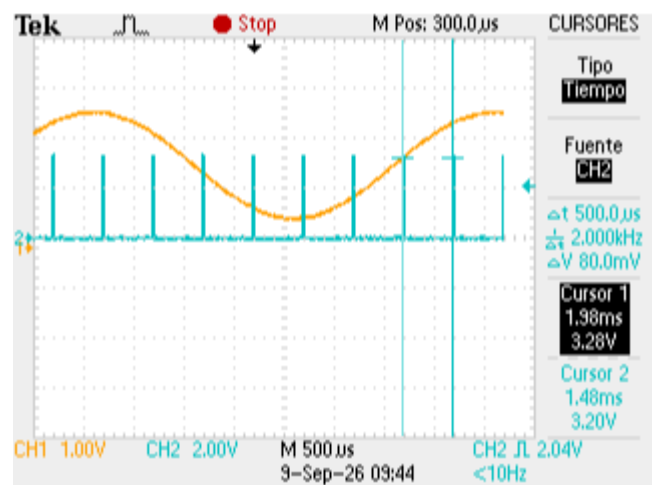


Figure 17. Periodo Muestreo 5

ANEXO C. Reportes de ejecución en consola

```
MPY: soft reboot
=====
ADC dynamic characterization - FFT
Raspberry Pi Pico 2 W - RP2350
=====

ADC input      : GP27
Sampling marker : GP15
N_SAMPLES (Ns) : 512
N_FFT (M)      : 1024
Target Fs      : 2000 Hz
Approx. physical df : 3.9063 Hz
Approx. FFT spacing : 1.9531 Hz

--- Test configuration ---
Input frequency from generator [Hz] (Enter = 250):
Window hann/rectangular (Enter = hann):

--- Sampling ---
Target Fs      : 2000.000 Hz
Measured Fs    : 2000.352 Hz
Fs error       : +0.0176 %
Mean Ts       : 499.912 us
Timing jitter (std) : 2.509 us
DC offset      : 1.647358 V

--- Frequency interpretation ---
Declared input frequency : 250.000 Hz
Measured Nyquist freq.   : 1000.176 Hz
Expected alias frequency : 250.000 Hz
FFT fundamental         : 250.043 Hz
FFT - expected alias     : +0.043 Hz
Samples per input period : 8.001
```

Figure 18. Reporte consola Parámetros de muestreo para la configuración con Ns = 512 y M = 1024

```
--- Spectral analysis ---
Window      : hann
Fundamental frequency : 250.043 Hz
Signal amplitude : 0.750344 Vrms
Signal amplitude : 1.061146 Vpeak
Signal amplitude : 2.122292 Vpp
Signal level    : -3.83 dBFS
Integrated noise : 0.005280 Vrms
Integrated noise : -46.89 dBFS
Measured FFT noise floor: -72.00 dBFS/bin
Ideal quantization noise: 0.000233 Vrms
Ideal SNR (12 bit) : 74.00 dB
Ideal FFT noise floor : -98.08 dBFS/bin
Window ENBW     : 1.500 bins
Window-adjusted floor : -96.32 dBFS/bin
SNR             : 43.05 dB
THD             : -60.00 dB (0.100 %)
THD+N          : -42.97 dB (0.711 %)
SINAD          : 42.97 dB
ENOB           : 6.84 bits
SFDR           : 47.40 dBc
Largest spur    : 232.463 Hz, -51.24 dBFS
Jitter-limited SNR est. : 48.09 dB

--- Output files ---
Time-domain data : adc_samples_250p0Hz_hann.csv
Frequency-domain data : adc_fft_250p0Hz_hann.csv
Summary          : adc_summary_250p0Hz_hann.csv
Analysis completed successfully.
```

Figure 19. Reporte consola Análisis espectral para la configuración con Ns = 512 y M = 1024

ANEXO D - Resultados consola análisis para diferentes tamaños de FFT

Anexo D1. M = 128

```
MPY: soft reboot
=====
ADC dynamic characterization - FFT
Raspberry Pi Pico 2 W - RP2350
=====

ADC input      : GP27
Sampling marker : GP15
N_SAMPLES (Ns) : 128
N_FFT (M)      : 128
Target Fs      : 2000 Hz
Approx. physical df : 15.6250 Hz
Approx. FFT spacing : 15.6250 Hz

--- Test configuration ---
Input frequency from generator [Hz] (Enter = 250):
Window hann/rectangular (Enter = hann):

--- Sampling ---
Target Fs      : 2000.000 Hz
Measured Fs    : 2001.545 Hz
Fs error       : +0.0772 %
Mean Ts       : 499.614 us
Timing jitter (std) : 4.650 us
DC offset      : 1.651958 V

--- Frequency interpretation ---
Declared input frequency : 250.000 Hz
Measured Nyquist freq.   : 1000.772 Hz
Expected alias frequency : 250.000 Hz
FFT fundamental         : 250.200 Hz
FFT - expected alias     : +0.200 Hz
Samples per input period : 8.006
```

Figure 20. Parámetros de muestreo e interpretación de frecuencia reportados por la consola para M = 128

```
--- Spectral analysis ---
Window      : hann
Fundamental frequency : 250.200 Hz
Signal amplitude : 0.926196 Vrms
Signal amplitude : 1.309839 Vpeak
Signal amplitude : 2.619679 Vpp
Signal level    : -2.01 dBFS
Integrated noise : 0.001153 Vrms
Integrated noise : -60.10 dBFS
Measured FFT noise floor: -69.45 dBFS/bin
Ideal quantization noise: 0.000233 Vrms
Ideal SNR (12 bit) : 74.00 dB
Ideal FFT noise floor : -92.06 dBFS/bin
Window ENBW     : 1.500 bins
Window-adjusted floor : -90.30 dBFS/bin
SNR             : 58.10 dB
THD             : -51.70 dB (0.260 %)
THD+N          : -50.80 dB (0.288 %)
SINAD          : 50.80 dB
ENOB           : 8.15 bits
SFDR           : 58.87 dBc
Largest spur    : 609.846 Hz, -60.88 dBFS
Jitter-limited SNR est. : 42.73 dB

--- Output files ---
Time-domain data : adc_samples_250p0Hz_hann.csv
Frequency-domain data : adc_fft_250p0Hz_hann.csv
Summary          : adc_summary_250p0Hz_hann.csv
Analysis completed successfully.
>>>
```

Figure 21. Resultados del análisis espectral y métricas dinámicas del ADC para M = 128.

Anexo D2. M = 256

```
MPY: soft reboot
=====
ADC dynamic characterization - FFT
Raspberry Pi Pico 2 W - RP2350
=====
ADC input      : GP27
Sampling marker : GP15
N_SAMPLES (Ns) : 256
N_FFT (M)      : 256
Target Fs      : 2000 Hz
Approx. physical df : 7.8125 Hz
Approx. FFT spacing : 7.8125 Hz

--- Test configuration ---
Input frequency from generator [Hz] (Enter = 250):
Window hann/rectangular (Enter = hann):

--- Sampling ---
Target Fs      : 2000.000 Hz
Measured Fs    : 2000.832 Hz
Fs error       : +0.0416 %
Mean Ts       : 499.792 us
Timing jitter (std) : 3.421 us
DC offset      : 1.651455 V

--- Frequency interpretation ---
Declared input frequency : 250.000 Hz
Measured Nyquist freq.   : 1000.416 Hz
Expected alias frequency : 250.000 Hz
FFT fundamental         : 250.105 Hz
FFT - expected alias    : +0.105 Hz
Samples per input period : 8.003
```

Figure 22. Parámetros de muestreo e interpretación de frecuencia reportados por la consola para M = 256

```
--- Spectral analysis ---
Window      : hann
Fundamental frequency : 250.105 Hz
Signal amplitude : 0.925668 Vrms
Signal amplitude : 1.309092 Vpeak
Signal amplitude : 2.618184 Vpp
Signal level    : -2.01 dBFS
Integrated noise : 0.003682 Vrms
Integrated noise : -50.02 dBFS
Measured FFT noise floor: -68.91 dBFS/bin
Ideal quantization noise: 0.000233 Vrms
Ideal SNR (12 bit) : 74.00 dB
Ideal FFT noise floor : -95.07 dBFS/bin
Window ENBW    : 1.500 bins
Window-adjusted floor : -93.31 dBFS/bin
SNR             : 48.01 dB
THD             : -55.45 dB (0.169 %)
THD+N          : -47.29 dB (0.432 %)
SINAD          : 47.29 dB
ENOB           : 7.56 bits
SFDR           : 56.65 dBc
Largest spur   : 148.499 Hz, -58.66 dBFS
Jitter-limited SNR est. : 45.40 dB

--- Output files ---
Time-domain data : adc_samples_250p0Hz_hann.csv
Frequency-domain data : adc_fft_250p0Hz_hann.csv
Summary          : adc_summary_250p0Hz_hann.csv
Analysis completed successfully.
```

Figure 23. Resultados del análisis espectral y métricas dinámicas del ADC para M = 256

Anexo D3. M = 512

```
MPY: soft reboot
=====
ADC dynamic characterization - FFT
Raspberry Pi Pico 2 W - RP2350
=====
ADC input      : GP27
Sampling marker : GP15
N_SAMPLES (Ns) : 512
N_FFT (M)      : 512
Target Fs      : 2000 Hz
Approx. physical df : 3.9063 Hz
Approx. FFT spacing : 3.9063 Hz

--- Test configuration ---
Input frequency from generator [Hz] (Enter = 250):
Window hann/rectangular (Enter = hann):

--- Sampling ---
Target Fs      : 2000.000 Hz
Measured Fs    : 2000.431 Hz
Fs error       : +0.0215 %
Mean Ts       : 499.892 us
Timing jitter (std) : 2.729 us
DC offset      : 1.652148 V

--- Frequency interpretation ---
Declared input frequency : 250.000 Hz
Measured Nyquist freq.   : 1000.215 Hz
Expected alias frequency : 250.000 Hz
FFT fundamental         : 250.052 Hz
FFT - expected alias    : +0.052 Hz
Samples per input period : 8.002
```

Figure 24. Parámetros de muestreo e interpretación de frecuencia reportados por la consola para M = 512

```
--- Spectral analysis ---
Window      : hann
Fundamental frequency : 250.052 Hz
Signal amplitude : 0.925758 Vrms
Signal amplitude : 1.309220 Vpeak
Signal amplitude | : 2.618440 Vpp
Signal level    : -2.01 dBFS
Integrated noise : 0.005027 Vrms
Integrated noise : -47.31 dBFS
Measured FFT noise floor: -70.17 dBFS/bin
Ideal quantization noise: 0.000233 Vrms
Ideal SNR (12 bit) : 74.00 dB
Ideal FFT noise floor : -98.08 dBFS/bin
Window ENBW    : 1.500 bins
Window-adjusted floor : -96.32 dBFS/bin
SNR             : 45.30 dB
THD             : -54.58 dB (0.187 %)
THD+N          : -44.82 dB (0.574 %)
SINAD          : 44.82 dB
ENOB           : 7.15 bits
SFDR           : 59.06 dBc
Largest spur   : 164.098 Hz, -61.08 dBFS
Jitter-limited SNR est. : 47.36 dB

--- Output files ---
Time-domain data : adc_samples_250p0Hz_hann.csv
Frequency-domain data : adc_fft_250p0Hz_hann.csv
Summary          : adc_summary_250p0Hz_hann.csv
Analysis completed successfully.
```

Figure 25. Resultados del análisis espectral y métricas dinámicas del ADC para M = 512

Anexo D4. M = 1024

```
MPY: soft reboot
=====
ADC dynamic characterization - FFT
Raspberry Pi Pico 2 W - RP2350
=====
ADC input      : GP27
Sampling marker : GP15
N_SAMPLES (Ns) : 1024
N_FFT (M)      : 1024
Target Fs      : 2000 Hz
Approx. physical df : 1.9531 Hz
Approx. FFT spacing : 1.9531 Hz

--- Test configuration ---
Input frequency from generator [Hz] (Enter = 250):
Window hann/rectangular (Enter = hann):

--- Sampling ---
Target Fs      : 2000.000 Hz
Measured Fs    : 2000.176 Hz
Fs error       : +0.0088 %
Mean Ts       : 499.956 us
Timing jitter (std) : 2.159 us
DC offset      : 1.651616 V

--- Frequency interpretation ---
Declared input frequency : 250.000 Hz
Measured Nyquist freq.   : 1000.088 Hz
Expected alias frequency : 250.000 Hz
FFT fundamental         : 250.021 Hz
FFT - expected alias    : +0.021 Hz
Samples per input period : 8.001
```

Figure 26. Parámetros de muestreo e interpretación de frecuencia reportados por la consola para M = 1024

```
--- Spectral analysis ---
Window      : hann
Fundamental frequency : 250.021 Hz
Signal amplitude : 0.924948 Vrms
Signal amplitude : 1.308074 Vpeak
Signal amplitude : 2.616148 Vpp
Signal level    : -2.02 dBFS
Integrated noise : 0.005448 Vrms
Integrated noise : -46.61 dBFS
Measured FFT noise floor: -73.03 dBFS/bin
Ideal quantization noise: 0.000233 Vrms
Ideal SNR (12 bit) : 74.00 dB
Ideal FFT noise floor : -101.09 dBFS/bin
Window ENBW    : 1.500 bins
Window-adjusted floor : -99.33 dBFS/bin
SNR             : 44.60 dB
THD             : -58.13 dB (0.124 %)
THD+N          : -44.41 dB (0.602 %)
SINAD           : 44.41 dB
ENOB            : 7.08 bits
SFDR            : 62.04 dBc
Largest spur    : 449.258 Hz, -64.06 dBFS
Jitter-limited SNR est. : 49.39 dB

--- Output files ---
Time-domain data : adc_samples_250p0Hz_hann.csv
Frequency-domain data : adc_fft_250p0Hz_hann.csv
Summary          : adc_summary_250p0Hz_hann.csv
Analysis completed successfully.
```

Figure 27. Resultados del análisis espectral y métricas dinámicas del ADC para M = 1024

ANEXO E. Visualización de señales y espectros MATLAB

ANEXO E1. M = 128

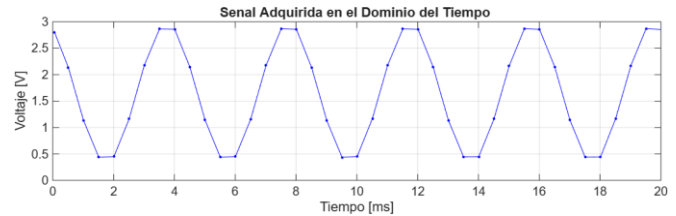


Figure 28. Señal en el dominio del tiempo (M=128).

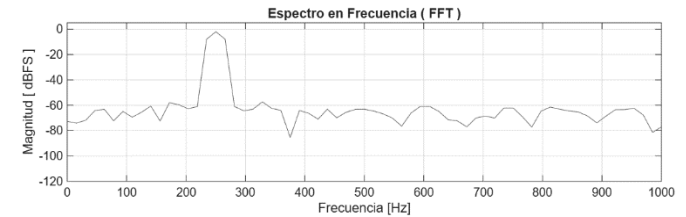


Figure 29. Espectro en frecuencia (M=128).

ANEXO E1. M = 256

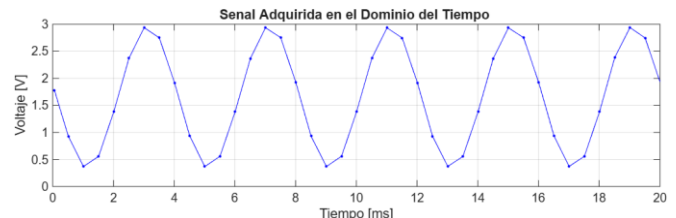


Figure 30. Señal en el dominio del tiempo (M=256).

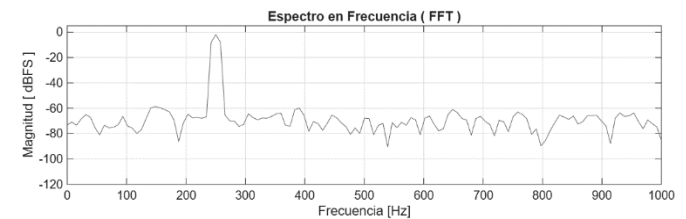


Figure 31. Espectro en frecuencia (M=256).

ANEXO E3. M = 512



Figure 32. Señal en el dominio del tiempo (M=512).

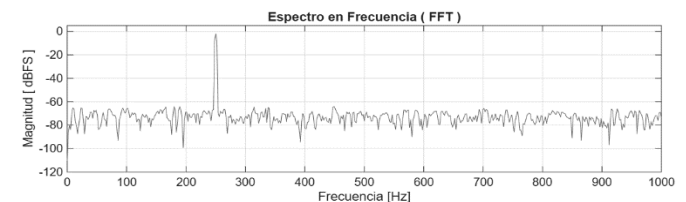


Figure 33. Espectro en frecuencia (M=512).

ANEXO E4. M= 1024

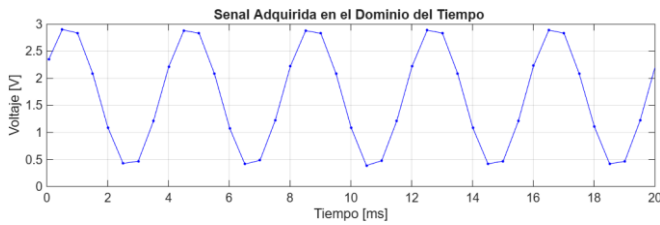


Figure 34. Señal en el dominio del tiempo (M=512).

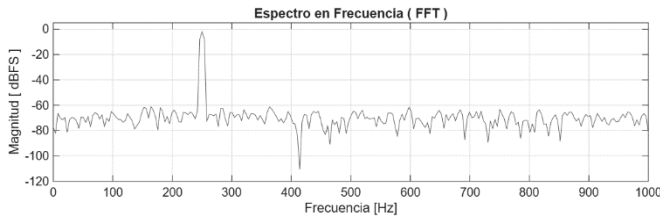


Figure 35. Espectro en frecuencia (M=512).

ANEXO F Código Superposición

```
%%
=====
%% Superposición en el Dominio de la
Frecuencia y del Tiempo
%% Pruebas A a D (M = 128, 256, 512, 1024)
%%
=====
clear; clc; close all;

try
    cd(fileparts(mfilename('fullpath')));
catch
    % Continuar si mfilename falla (ej. al
    ejecutar por selección)
end

% --- 1. Definir los archivos y etiquetas de
cada prueba ---
archivos = { ...
    'adc_fft_M128.csv', ...
    'adc_fft_M256.csv', ...
    'adc_fft_M512.csv', ...
    'adc_fft_M1024.csv' ...
};
etiquetas = {'M = 128', 'M = 256', 'M = 512',
'M = 1024'};
colores = {[0, 0.45, 0.74], [0.85, 0.33,
0.10], [0.47, 0.67, 0.19], [0.49, 0.18,
0.56]};

% --- 2. Cargar datos en memoria una sola vez
---
num_archivos = length(archivos);
datos = cell(1, num_archivos);
for k = 1:num_archivos
    datos{k} = readtable(archivos{k});
end

% Dominio del tiempo
archivos_tiempo = { ...
    'adc_time_M128.csv', ...
    'adc_time_M256.csv', ...
    'adc_time_M512.csv', ...
    'adc_time_M1024.csv' ...
};
datos_tiempo = cell(1, num_archivos);
for k = 1:num_archivos
    datos_tiempo{k} =
readtable(archivos_tiempo{k});
end

% --- 3. Graficar todos los espectros
superpuestos (Dominio de Frecuencia) ---
figure('Color', 'w', 'Name', 'Superposición
Espectros FFT');
hold on;
for k = 1:num_archivos
    plot(datos{k}.frequency_Hz,
datos{k}.magnitude_dBFS, ...
        'Color', colores{k}, 'LineWidth',
1.1, ...
        'DisplayName', etiquetas{k});
end
grid on;
xlabel('Frecuencia [Hz]');
ylabel('Magnitud [dBFS]');
title('Superposición de Espectros FFT -
Efecto de M sobre el Piso de Ruido');
xlim([0, 1000]); % Banda de Nyquist (Fs/2 =
1000 Hz)
ylim([-120, 5]);
legend('Location', 'northeast');
hold off;

% --- 4. Graficar señales superpuestas
(Dominio del Tiempo) ---
figure('Color', 'w', 'Name', 'Superposición
Dominio del Tiempo');
hold on;
etiqueta_y = 'Amplitud';

% Graficamos en orden inverso (de mayor a
menor M) para que M=128 quede encima
for k = num_archivos:-1:1
    df = datos_tiempo{k};

    if ismember('voltage_V',
df.Properties.VariableNames)
        eje_y = df.voltage_V;
```



```
        etiqueta_y = 'Voltaje [V]';
    elseif ismember('sample_val',
df.Properties.VariableNames)
        eje_y = df.sample_val;
        etiqueta_y = 'Muestra Digital [LSB]';
    else
        eje_y = df{:, 2};
        etiqueta_y = 'Amplitud';
    end

    plot(df.time_s, eje_y, ...
        'Color', colores{k}, 'LineWidth',
1.2, ...
        'DisplayName', etiquetas{k});
end

grid on;
xlabel('Tiempo [s]');
ylabel(etiqueta_y);
title('Superposición de Señales en el Dominio
del Tiempo');
legend('Location', 'northeast');

% CLAVE: Hacer zoom en los primeros ciclos
para ver la onda claramente
xlim([0, 0.015]); % Muestra aproximadamente
15 ms (ajusta según la frecuencia de tu
señal)
hold off;
```